

Taller ABP - Entrega 1: Ecosistema Interactivo Web (20%)

Curso: Lenguajes de Programación para la Web

Metodología: Aprender Haciendo

Modalidad: Parejas o Individual

1. Datos del Proyecto y Asistente Metodológico Web

Título Formal del Proyecto Web:

 **Qué es:** Nombre técnico que delimita la aplicación web, la arquitectura de software (Full-Stack, SPA, MVC) y el problema a resolver.

 **Ejemplos Asistidos (haz clic para usar):**

Sistema de Control de Inventarios de Activos TI

Integrantes del Equipo:

 **Qué es:** Nombres completos de los desarrolladores Full-Stack / Frontend / Backend del proyecto.

Cristian Alejandro Bustamante Escobar

Selección de Metodología de Desarrollo Web:

 **Qué es:** Marco de trabajo adaptable al ciclo de vida del software web (Modelado BD, endpoints Backend, interfaces Frontend e integración CI/CD).

MVP Web Iterativo

Web MVP Iterativo: Creación acelerada del core backend y vistas básicas para validación rápida de navegación y consumo de datos.

Enlace al Repositorio GitHub y/o Despliegue en Vivo (URL):

 **Qué es:** URLs del código fuente en GitHub (Branch principal con commits claros) y el enlace público de producción (Vercel, Render, Netlify).

<https://github.com/A-Bustamante/ProyectoLPW.git>

2. Planteamiento del Problema y Pregunta Problema Web

Contexto y Asistente de Diagnóstico del Problema Web:

■ **Qué es:** Descripción de las deficiencias actuales en los sistemas analizados (procesos manuales en papel, páginas estáticas sin persistencia, vulnerabilidades de seguridad o mala adaptabilidad responsive).

💡 **Plantillas rápidas para cargar en el cuadro:**

Actualmente el control de los activos de TI se lleva en hojas de excel sin mayor control y no se tiene muy claro lo que hay disponible para asignación

Formulación de la Pregunta Problema:

■ **Qué es:** La pregunta de ingeniería de software que guía el desarrollo del sistema web.

💡 **Ejemplo:** "¿De qué manera el desarrollo de una aplicación web interactiva con arquitectura RESTful y backend persistente permite optimizar en un 50% el tiempo de procesamiento de solicitudes?"

¿De que manera podemos desarrollar una aplicación que permita tener un mejor control de los activos de TI?

3. Constructor Asistido de Objetivos SMART

Objetivo General (SMART):

■ **Qué es:** Meta global del proyecto web. Debe indicar tecnologías (Frontend/Backend/BD), métrica de rendimiento/usabilidad y tiempo de entrega.

💡 **Haz clic para cargar plantilla SMART:**

Diseñar y desarrollar una aplicación web Full-Stack utilizando Django, Jazzmin, Node.js y PostgreSQL, garantizando normas internacionales de desarrollo y solucionando el problema actual

Objetivos Específicos (Estructura Metodológica Web de 3 Pasos):

■ **Qué es:** La ruta técnica lógica: 1. Diseñar arquitectura y BD -> 2. Programar Backend/Frontend -> 3. Pruebas y Despliegue en la Nube.

Paso 1: Modelado de Datos y Arquitectura Web

1. Diseñar arquitectura y BD

Paso 2: Programación Backend (API) e Interfaz Frontend

2. Programar Backend/Frontend

Paso 3: Pruebas de Software, Seguridad y Despliegue Cloud

3. Pruebas y Despliegue en la Nube.

4. Arquitectura y Simulador de Costos Web (Hosting + Dominios + BD)

Arquitectura del Stack Tecnológico Seleccionado:

■ **Qué es:** Definición del Stack de desarrollo (MERN, PERN, Django+React, LAMP, Spring Boot).

Python Web Architecture (Django / FastAPI + HTML5/Tailwind / JS Vanilla)

Simulador de Costos de Infraestructura Web, Hosting y Dominios:

■ **Qué es:** Calculadora para estimar los costos mensuales y anuales de publicar la aplicación web (Dominio .com/.co, Hosting/VPS, Servidor de Base de Datos y Certificados SSL).

Servicio / Recurso Web	Proveedor Cloud	Frecuencia / Detalle	Costo USD	Subtotal USD	Acción
------------------------	-----------------	----------------------	-----------	--------------	--------

Costo Total Estimado Proyecto Web:

\$0.00 USD

Presupuesto Asignado:

0

USD

Eficiencia de Presupuesto:

100.0% libre

Estrategia de Seguridad Web (JWT, CORS, Sanitización SQL/XSS, HTTPS):

■ **Qué es:** Medidas implementadas para proteger la aplicación web frente a ataques comunes (protección contra Inyección SQL mediante ORM/Prepared Statements, prevención de XSS, políticas CORS y autenticación con Tokens JWT).

Detalle las técnicas de seguridad aplicadas en la API y el Frontend...